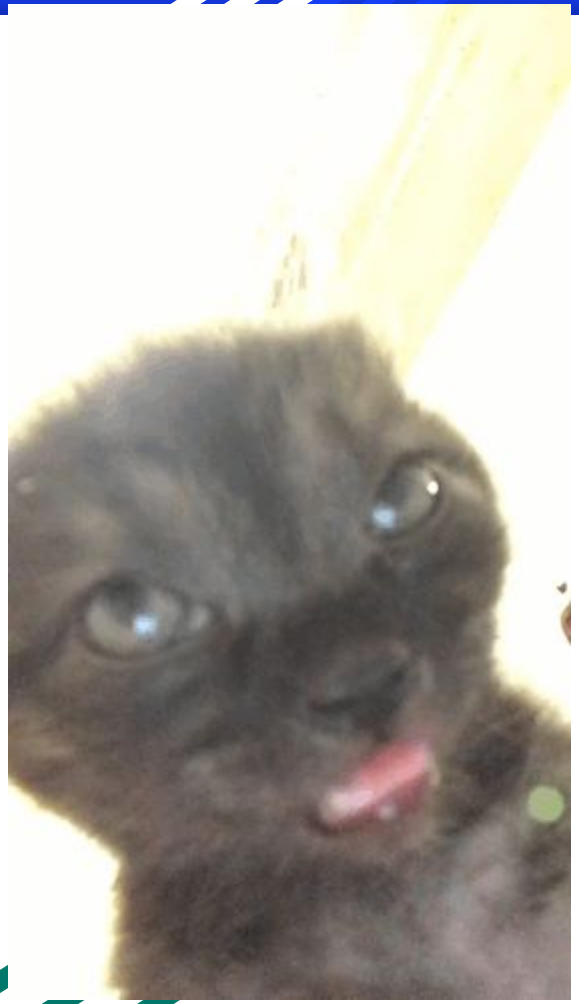
Tutorial 10

2D arrays and Abstract types




eepy

Overview

2D Arrays: Static vs Dynamic

- Declaration
- Initialization
- Access
- Example: calendar

Abstract type

- Syntax, declaration, initialization
- Arrays for structs

2D Arrays: Declaration

- Static
  - Stack: Memory is allocated at compile time
  - The array's size cannot be changed when the program is in execution.
  - Declaration:
    - 1D: int array[3];
    - 2D: int array[3][5] ;
- Dynamic
  - Heap: Memory is allocated at run time
  - The array's size can be changed when the program is in execution.
  - Declaration:
    - 1D: int *array = new int[3]; //Memory must be delete
    - 2D:
      - int **array = new int*[3];
      - for(int i=0 ; i<3 ; i++) array[i] = new int[5];
      - for(int i=0 ; i<3 ; i++) *(array+i) = new int[5];

2D Arrays: Initialization

- Static
  - Can initialize array elements at the time of declaration
  - 1D:
    - int array[3] = {1,2,3}
    - int array[] = {1,2,3}
  - 2D:
    - int array[3][2] = { {1,2}, {3,4},{5,6}}
    - int array[][2] = { {1,2}, {3,4},{5,6}}
- Dynamic
  - Cannot initialize array elements at declaration,
  - 1D: int *array = new int[size] //empty array or array initialized with garbage values

2D Arrays: Access

Both static and dynamic arrays can be accessed using either

- Square brackets:
  - 1D: array_name[index]
  - 2D: array_name[row_index][column_index]
- Pointer arithmetic:
  - 1D: *(array_name + index)
  - 2D: *(*(array_name + row_index) + column_index)

Exercise, access and initialize the following arrays at row 2 column 2

- Static: int array[2][2]
- Dynamic:

```
int **array = new int*[2];
for( int i=0; i<2 ; i++ )
        array[i] = new int[2]
```

2D Arrays: Exercise

- Declare and initialize a calendar as a 2D array of strings
  - months x days
  - Note: months have different number of days
  - Solution:
    - Static: all months have same number of days
    - Dynamic: every month is allocated the exact number of days as in reality.
- Homework:
  - Declare a 2D dynamic array to record the seating arrangement for COS 132 exam
    - 4 lecture halls with capacity: 150, 250, 200 and 160
  - Allocate the seats to the 700 students whose student numbers are stored in array
    - string classlist[700] = { “u12345678”, ….., “u12346789”}

Abstract types: Syntax, declaration, initialization

- Syntax

```
struct name {
    datatype name1;
    datatype name2;
    datatype2 name3;
}
```

- Declaration
  - name mystruct;
- Initialization
  - mystruct.name1 = val1 ;
  - mystruct.name3 = val3 ;
- Declaration + initialization
  - name mystruct = { val1, val2, val3 };

Example

```
struct studentInfo{
        string name;
        string surname;
        float st1;
        float st2;
        float quizzes;
        float practical;
        string getFullname();
};

studentInfo student1 = { “Thabo”, “Zulu” };
```

Abstract types: arrays of structs

Exercise

- Declare an array of 10 structures of type studentInfo
- Initialize the 10 studentInfo via user input
- Search and find the student with the highest marks as a sum of the 4 assessments completed through the semester
- Calculate the averages for the different assessments and print them out

END